

```
In [1]: candidate_profile = """
Rahul Sharma is a highly motivated Computer Science graduate currently
pursuing a Bachelor of Engineering in Computer Science with an excellent
academic record and a strong interest in Artificial Intelligence,
Machine Learning, Cloud Computing, and Full Stack Web Development. He
possesses technical skills in Python, Java, C++, JavaScript, React,
Node.js, Spring Boot, MySQL, MongoDB, Git, Docker, AWS, HTML, CSS, and
REST APIs. During his internship at an IT services company, he developed
backend APIs using Spring Boot, designed database schemas in MySQL,
fixed application bugs, collaborated with senior developers, and
participated in Agile sprint meetings. Rahul completed several academic
and personal projects including an AI-based Resume Screening System,
Employee Attendance Management System, Smart Library Management System,
Weather Forecast Web Application, and an Online Shopping Portal. He also
developed an AI chatbot for answering student queries using natural
language processing techniques. He earned certifications in AWS Cloud
Practitioner, Oracle Java Programming, Google Data Analytics, Python for
Everybody, and Introduction to Machine Learning. Rahul enjoys learning
new technologies, solving programming challenges, contributing to open-
source projects, participating in hackathons, and continuously improving
his technical knowledge. His career goal is to become an AI Engineer who
designs scalable intelligent applications capable of solving real-world
business problems while continuously learning emerging technologies and
contributing to innovative software solutions.
"""
```

```
In [2]: MAX_TOKEN_LIMIT = 320

words = candidate_profile.split()

word_count = len(words)

estimated_tokens = word_count * 1.25

print("Original Word Count :", word_count)
print("Estimated Tokens      :", estimated_tokens)

if estimated_tokens <= MAX_TOKEN_LIMIT:
    print("Status : Within the model context limit.")
else:
    print("Status : Exceeds the model context limit.")
```

```
Original Word Count : 213
Estimated Tokens      : 266.25
Status : Within the model context limit.
```

```
In [3]: optimized_profile = """
Rahul Sharma is pursuing a Bachelor of Engineering in Computer Science
with strong knowledge of Artificial Intelligence and Full Stack
Development. His core technical skills include Python, Java, C++,
JavaScript, React, Spring Boot, MySQL, MongoDB, Git, Docker, AWS, HTML,
CSS, and REST APIs. During his internship, he developed backend APIs,
designed MySQL databases, fixed bugs, and collaborated in Agile teams.
His key projects include an AI Resume Screening System, Employee
Attendance Management System, Smart Library Management System, and an AI
chatbot using natural language processing. He holds AWS Cloud
Practitioner, Oracle Java Programming, Google Data Analytics, Python for
Everybody, and Introduction to Machine Learning certifications. His
career objective is to become an AI Engineer who develops scalable
intelligent applications for real-world business challenges.
"""
```

```
In [4]: optimized_words = optimized_profile.split()

optimized_word_count = len(optimized_words)

optimized_tokens = optimized_word_count * 1.25

tokens_saved = estimated_tokens - optimized_tokens

reduction_percentage = (tokens_saved / estimated_tokens) * 100

print("Original Word Count      :", word_count)
print("Original Tokens          :", estimated_tokens)

print()

print("Optimized Word Count      :", optimized_word_count)
print("Optimized Tokens          :", optimized_tokens)

print()

print("Tokens Saved                :", tokens_saved)
print("Reduction Percentage (%)    :", round(reduction_percentage,2))
```

```
Original Word Count      : 213
Original Tokens          : 266.25

Optimized Word Count     : 123
Optimized Tokens         : 153.75

Tokens Saved             : 112.5
Reduction Percentage (%) : 42.25
```

```
In [5]: import pandas as pd

comparison = pd.DataFrame({
    "Input Version":[
        "Original Candidate Profile",
        "Optimized Candidate Profile"
    ],
    "Word Count":[
        word_count,
        optimized_word_count
    ],
    "Estimated Tokens":[
        estimated_tokens,
        optimized_tokens
    ],
    "Maximum Token Limit":[
        MAX_TOKEN_LIMIT,
        MAX_TOKEN_LIMIT
    ],
    "Limit Status":[
        "Within Limit" if estimated_tokens<=MAX_TOKEN_LIMIT else
"Exceeded",
        "Within Limit" if optimized_tokens<=MAX_TOKEN_LIMIT else
"Exceeded"
    ],
    "Tokens Saved":[
        0,
        tokens_saved
    ]
})

comparison
```

Out[5]:

	Input Version	Word Count	Estimated Tokens	Maximum Token Limit	Limit Status	Tokens Saved
0	Original Candidate Profile	213	266.25	320	Within Limit	0.0
1	Optimized Candidate Profile	123	153.75	320	Within Limit	112.5

```
In [6]: print("1. Why is token estimation important?")
print("Token estimation helps predict whether the input fits within the
model's context window before sending it to an LLM. It prevents request
failures and improves prompt planning.")

print("\n2. Problems when token limit is exceeded:")
print("- Request rejection")
print("- Prompt truncation")
print("- Important candidate details may be omitted")
print("- Higher processing time")

print("\n3. How does prompt optimization improve efficiency?")
print("Prompt optimization reduces unnecessary text, lowers token usage,
speeds up responses, and reduces API costs while preserving essential
information.")

print("\n4. Information that should always be preserved:")
print("- Highest qualification")
print("- Technical skills")
print("- Internship or work experience")
print("- Major projects")
print("- Certifications")
print("- Career objective")

print("\n5. How do unnecessary tokens increase API cost?")
print("Most LLM providers charge based on tokens. Extra words increase
input tokens, leading to higher costs and slower processing in large
recruitment systems.")

print("\n6. Why retain important keywords?")
print("Important keywords such as programming languages, frameworks,
certifications, and qualifications help the LLM accurately understand
candidate capabilities and improve recruitment decisions.")
```

1. Why is token estimation important?

Token estimation helps predict whether the input fits within the model's context window before sending it to an LLM. It prevents request failures and improves prompt planning.

2. Problems when token limit is exceeded:

- Request rejection
- Prompt truncation
- Important candidate details may be omitted
- Higher processing time

3. How does prompt optimization improve efficiency?

Prompt optimization reduces unnecessary text, lowers token usage, speeds up responses, and reduces API costs while preserving essential information.

4. Information that should always be preserved:

- Highest qualification
- Technical skills
- Internship or work experience
- Major projects
- Certifications

- Career objective

5. How do unnecessary tokens increase API cost?

Most LLM providers charge based on tokens. Extra words increase input tokens, leading to higher costs and slower processing in large recruitment systems.

6. Why retain important keywords?

Important keywords such as programming languages, frameworks, certifications, and qualifications help the LLM accurately understand candidate capabilities and improve recruitment decisions.

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).